FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Ejemplos de Desnormalización

Gestión de Bases de Datos

Rodrigo Pavez Madariaga – rpavez@utalca.cl

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Desnormalización en un *e-commerce*

Considere el siguiente escenario.

Se tiene una base de datos normalizada (3FN) con las siguientes tablas:

- **CLIENTE**(**id_cliente**, nombre, ciudad, segmento)
- **PRODUCTO**(**id_producto**, nombre, categoría, precio)
- **PEDIDO**(**id_pedido**, ***id_cliente***, fecha)
- **DETALLE_PEDIDO**(**id_detalle**, ***id_pedido***, ***id_producto***, cantidad, precio_unitario)

Y se sabe que el sistema ha crecido considerablemente:

- CLIENTE: **2 millones de registros**
- PRODUCTO: **500 mil registros**
- PEDIDO: **50 millones de registros**
- DETALLE_PEDIDO: **300 millones de registros**

Desnormalización en un *e-commerce*

Ejemplo de Datos

CLIENTE			
id_cliente	**nombre**	**ciudad**	**segmento**
1	Ana	Santiago	Premium
2	Luis	Valparaíso	Regular

PEDIDO		
id_pedido	**id_cliente**	**fecha**
100	1	10-01-25
101	2	15-02-25

PRODUCTO			
id_producto	**nombre**	**categoria**	**precio**
10	Laptop	Electrónica	800000
20	Polera	Ropa	15000

DETALLE_PEDIDO				
id_detalle	**id_pedido**	**id_producto**	**cantidad**	**precio_unitario**
1	100	10	1	800000
2	100	20	2	15000
3	101	20	3	15000

Desnormalización en un *e-commerce*

El sistema de *analítica* y *dashboard* ejecuta constantemente consultas como:

- **Ventas totales por categoría y mes**
- **Top 10 productos más vendidos por ciudad**
- **Ingresos por segmento de cliente**
- **Histórico de compras de un cliente (con detalle de productos)**

Desnormalización en un *e-commerce*

Ejemplo de consulta típica

Resumen de ventas agrupado por ciudad y categoría de producto dentro de un rango de fechas específico

```
SELECT c.ciudad, p.categoria, SUM(dp.cantidad * dp.precio_unitario) AS total
FROM CLIENTE c
JOIN PEDIDO pe ON c.id_cliente = pe.id_cliente
JOIN DETALLE_PEDIDO dp ON pe.id_pedido = dp.id_pedido
JOIN PRODUCTO p ON dp.id_producto = p.id_producto
WHERE pe.fecha BETWEEN '2025-01-01' AND '2025-03-31'
GROUP BY c.ciudad, p.categoria;
```

Desnormalización en un *e-commerce*

Problemas detectados

Luego de una auditoría al sistema, dado que estaba demorando demasiado en responder en algunas áreas, se evidencian las siguientes situaciones:

- La consulta más frecuente requiere **múltiples JOINs sobre tablas masivas**
- Tiempo de respuesta: **8–15 segundos por consulta**
- Alta carga en el servidor (especialmente en horarios de reporte)
- Uso intensivo de CPU y disco

Desnormalización en un *e-commerce*

Propuesta de Desnormalización

Con la idea de optimizar los tiempos de respuesta de la base de datos, se establece que una buena alternativa sería desnormalizar algunas tablas del sistema.

¿Cuál sería su propuesta de desnormalización?

¿Cómo se justificaría?

Desnormalización en un *e-commerce*

Estrategia de Desnormalización

Se decide crear una tabla desnormalizada donde se dupliquen datos de CLIENTE y PRODUCTO, en una nueva tabla llamada HECHO_VENTAS, con la siguiente estructura:

- **id_pedido**
- fecha
- id_cliente
- ciudad_cliente
- segmento_cliente
- **id_producto**
- categoria_producto
- cantidad
- precio_unitario
- total_linea

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Desnormalización en un *e-commerce*

Datos de la Tabla Desnormalizada

HECHO_VENTAS									
id_pedido	fecha	id_cliente	ciudad_cliente	segmento_cliente	id_producto	categoria_producto	cantidad	precio_unitario	total_linea
100	10-01-25	1	Santiago	Premium	10	Electrónica	1	800000	800000
100	10-01-25	1	Santiago	Premium	20	Ropa	2	15000	30000
101	15-02-25	2	Valparaíso	Regular	20	Ropa	3	15000	45000

Consulta Optimizada

SELECT ciudad_cliente, categoria_producto, **SUM**(total_linea) **AS** total
FROM *HECHO_VENTAS*
WHERE fecha **BETWEEN** '2025-01-01' **AND** '2025-03-31'
GROUP BY ciudad_cliente, categoria_producto;

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Desnormalización en un *e-commerce*

¿Se responde a las 4 consultas frecuentes?

Ventas totales por categoría y mes

Mejora: MUY ALTA

Por qué: Ya tienes en la tabla desnormalizada:

- categoria_producto
- fecha
- total_linea

Evitas JOINs con PRODUCTO, PEDIDO y DETALLE

Resultado:

Consulta directa sobre una sola tabla

Ideal para agregar (SUM) y agrupar (GROUP BY)

- Este es el caso **más beneficiado**
- Puede pasar de segundos → milisegundos

Desnormalización en un *e-commerce*

¿Se responde a las 4 consultas frecuentes?

Top 10 productos más vendidos por ciudad

Mejora: ALTA

Por qué: Ya tienes:

- ciudad_cliente
- id_producto
- cantidad

Ventaja:

- Eliminación de JOIN con CLIENTE y PEDIDO
- Solo agregas y ordenas

Posible mejora adicional:

- Si agregas nombre_producto, evitas incluso JOIN con PRODUCTO

- Muy buen rendimiento
- Especialmente útil si se ejecuta frecuentemente

Desnormalización en un *e-commerce*

¿Se responde a las 4 consultas frecuentes?

Ingresos por segmento de cliente

Mejora: MUY ALTA

Por qué:

Campo ya desnormalizado: segmento_cliente

Uso directo de total_linea

- Consulta extremadamente eficiente
- Similar al caso 1

Desnormalización en un *e-commerce*

¿Se responde a las 4 consultas frecuentes?

Histórico de compras de un cliente (con detalle de productos)

Mejora: MEDIA

Por qué: Ya tienes:

- id_cliente
- producto
- cantidad
- precio
- fecha

Pero ojo: Este tipo de consulta es más **transaccional (OLTP)** que analítica

Puede requerir:

- más detalles del producto (descripción, stock, etc.)
- ordenamientos y paginación

Conclusión:

- Mejora, pero **no es el principal objetivo de la desnormalización**
- En algunos casos, una tabla normalizada bien indexada puede rendir igual o mejor

Desnormalización en un *e-commerce*

Resultados del proceso

- Tiempo de respuesta: **< 1 segundo**
- Eliminación de **JOINs** complejos
- Mejora significativa en ***dashboards*** y ***reportes***
- Reducción de carga en CPU

Desnormalización en un *e-commerce*

Justificación de la desnormalización

Alta frecuencia de consultas analíticas

- Consultas ejecutadas cientos o miles de veces al día

Volumen de datos muy grande

- JOINs sobre cientos de millones de registros

Datos relativamente estables

- Ciudad y categoría cambian muy poco

Optimización de lectura sobre escritura

- Sistema orientado a reportes (OLAP-like)

Desnormalización en un *e-commerce*

Costos de la desnormalización

- ***Redundancia*** de datos
- ***Mayor espacio*** en disco
- Necesidad de procesos ***ETL*** o ***triggers*** para mantener consistencia
- Riesgo de ***inconsistencias*** si no se gestiona correctamente

Desnormalización: sacrifica consistencia controlada para ganar rendimiento